Guía N° 1 — Boot de Micro S.O. en QEMU

INFB6074 Infraestructura para Ciencia de Datos — UTEM, 2.º semestre 2026

Felipe Canales Huerta · Docente: Dr. Ing. Michael Miranda Sandoval · Noviembre de 2026

1. Propósito y entorno

La guía construye, en tres pasos, la cadena que lleva desde un archivo de texto hasta una máquina que arranca: un sector de arranque de 512 bytes, un cargador en dos etapas con menú y un kernel en C que dibuja píxeles. Cada paso se ensambla o compila desde fuentes versionados, produce un artefacto binario verificable (sector, imagen de disquete, ISO) y se ejecuta en una máquina virtual. Las capas de infraestructura que en ciencia de datos suelen quedar ocultas (imagen, cargador, kernel, hardware virtual) quedan así a la vista.

Entorno. Windows 11 (10.0.26300) como anfitrión. NASM 3.01 y QEMU 11.0.0 (`qemu-system-i386`) nativos para las fases 1 y 2. Para la Fase 3, un contenedor Docker con Debian 12: gcc 12.2.0, binutils 2.40, NASM 2.16.01, GRUB 2.06 y xorriso 1.5.4. Git 2.54 para el versionado y Python 3 (solo biblioteca estándar) para la conversión de la imagen y las capturas automáticas. Diferencia relevante del entorno: en Windows no hay `grub-mkrescue` y el gcc de MSYS2 genera objetos PE, por lo que la Fase 3 se construye en el contenedor.

Evidencias reproducibles. Las capturas no son pantallazos manuales. `herramientas/captura_qemu.py` arranca QEMU sin ventana, envía teclas por el monitor (`sendkey`) y guarda la memoria de video emulada con `screendump`. `herramientas/generar_evidencias.sh` reconstruye las tres fases y vuelve a tomar todas las capturas, de modo que siempre corresponden al código del repositorio.

2. Fase 1: sector de 512 bytes y dibujo ASCII

Flujo fuente → ensamblado → arranque. `boot_f1.asm` se ensambla con `nasm -f bin`, que produce un binario plano sin cabeceras: los bytes del archivo son exactamente las instrucciones y datos que verá la CPU. QEMU presenta ese archivo como un disquete. El BIOS (SeaBIOS) lee su primer sector, comprueba la firma `55 AA` en los bytes 510–511, lo copia a `0000:7C00` y salta a él en modo real de 16 bits. Por eso el fuente declara `org 0x7C00` y termina con `times 510-($-$$) db 0` y `dw 0xAA55`. Si el programa excede 510 bytes, NASM falla al ensamblar. Además, `build.sh` comprueba después el tamaño (512 bytes) y la firma.

Dibujo. El caballo al galope (diseño propio, 14×39 caracteres, con crin y cola en café y pasto en verde) se guarda como un flujo de bytes con un formato propio: `0x0A` es un salto de línea, `0x01`–`0x1F` avanza N columnas, `0x80`–`0xFF` cambia el color y `0x00` termina. Los espacios iniciales y los colores cuestan un byte cada uno, y el programa completo ocupa 360 de los 510 bytes. Cada carácter se escribe con `INT 10h AH=09h` (carácter + atributo) y el cursor se mueve con `AH=02h`.

Errores y decisiones. El ejemplo de la guía usa el teletipo `INT 10h AH=0Eh`, que en modo texto no admite colores. Se reemplazó por la combinación `AH=09h` + `AH=02h`, a cambio de avanzar la columna por software. También se fijó el modo de video `03h` al inicio, lo que limpia la pantalla del BIOS y hace que el resultado no dependa del estado previo.

3. Fase 2: mapa del disco, segunda etapa y menú

Mapa del disco. La imagen es un disquete de 1,44 MB (2880 sectores, CHS 80×2×18). El LBA 0 contiene la etapa 1 (512 B con firma); los LBA 1–16 (CHS 0/0/2–0/0/17), la etapa 2, con 16 sectores (8 KiB) reservados de los cuales usa 1968 B; el resto son ceros. Las ubicaciones están en un solo archivo, `src/disposicion.inc`, que incluyen ambas etapas y del que el script de construcción lee `ETAPA2_LBA` y `ETAPA2_SECTORES` para colocar la etapa con `dd`. Así no se pueden desincronizar.

Traspaso de control. La etapa 1 guarda la unidad de arranque que entrega el BIOS en `DL` y lee los 16 sectores con `INT 13h AH=02h` (CHS: cilindro 0, cabeza 0, sector 2), con hasta 3 intentos y `AH=00h` (reinicio de la controladora) entre ellos. Se eligió CHS porque el dispositivo emulado es un disquete y la etapa cabe en la primera pista. Una directiva `%error` impide ensamblar si dejara de caber. Antes de saltar, la etapa 1 comprueba que los 4 primeros

bytes leídos sean la firma `ETP2`. Luego restablece `DL` y hace `jmp 0000:8004`. La etapa 2 recibe la CPU en modo real con `CS = DS = ES = SS = 0`, `SP = 0x7C00` e interrupciones habilitadas.

Lectura de teclado y menú. La etapa 2 muestra una barra con su dirección de carga, el LBA y la unidad; el caballo de la Fase 1, y un menú. `INT 16h AH=00h` entrega el ASCII en `AL` y el código de rastreo en `AH`. Cada opción da una respuesta distinta: (1) información del sistema con `INT 12h`, `INT 15h AH=88h` e `INT 10h AH=0Fh`; (2) una cuenta regresiva y el galope del caballo, que se redibuja dos columnas más a la derecha en cada cuadro (lo que sale de la columna 79 se recorta) y mide el tiempo con el contador de ticks del BIOS en `0040:006C`; (3) fecha y hora del RTC con `INT 1Ah` (en BCD, que se lee directo en hexadecimal), y (0) detención controlada. Una tecla no válida muestra en rojo el carácter y su código, y se vuelve a pedir la opción sin reiniciar.

Tabla de pruebas (resumen; detalle y capturas en `fase-2/README.md`).

Caso	Acción	Esperado	Observado
Arranque normal	arrancar `disco_f2.img`	etapa 1 informa la carga; etapa 2 con caballo y menú	igual a lo esperado
Opción 1	tecla `1`	datos del sistema	639 KB, 64512 KB, modo 0x03 de 80 columnas, 1968 B
Opción 2	tecla `2`	cuenta 3-2-1 y animación	igual a lo esperado
Opción 3	tecla `3`	fecha y hora del RTC	hora UTC del anfitrión
Tecla inválida	`x`, `F1`	aviso y reintento	`'x' (0x2D)`, `(0x3B)`; el menú sigue respondiendo
Opción 0	tecla `0`	detención controlada	igual a lo esperado
Fallo controlado	disco sin etapa 2	error de firma y detención	`Firma ETP2 no encontrada`
Disposición	`build.sh`	512 B + `55AA`; `ETP2` en el byte 512	verificado por el script

Análisis del fallo. Con el disco sin etapa 2, `INT 13h` *no* informa error, porque los sectores existen y contienen ceros. Sin la firma, la etapa 1 saltaría a ejecutar ceros sin aviso. La comprobación convierte un fallo silencioso en uno visible. **Errores corregidos:** el código de error de `INT 13h` se perdía porque la rutina de impresión reutiliza `AH` (se guardó en la pila); los mensajes de la etapa 1 desaparecían al instante (se agregó una pausa de ~2 s), y el título de la etapa 2 pasaba de 80 columnas.

4. Fase 3: cargador, kernel y framebuffer

Cargador e interfaz. Siguiendo la recomendación de la guía, se usa GRUB 2.06 con Multiboot2. `boot_f3.asm` contiene la cabecera Multiboot2 (magic `0xE85250D6`, arquitectura i386, suma de verificación y una etiqueta de framebuffer que pide 800×600×32, marcada como opcional) y el punto de entrada `_start`. GRUB carga el ELF en 1 MiB, carga la imagen como módulo, activa el modo VBE, pasa a modo protegido de 32 bits y salta a `_start` con `EAX = 0x36D76289` y `EBX` apuntando a la información Multiboot2. `_start` crea una pila de 16 KiB y llama a `kmain(magic, info)`.

Kernel freestanding. El kernel está escrito en C11 y se compila con `gcc -m32 -march=i686 -ffreestanding -fno-pie -nostdlib -mgeneral-regs-only`. Se enlaza con `ld -m elf_i386` y un `linker.ld` propio que deja la cabecera en los primeros 32 KiB. No depende de ninguna biblioteca del sistema: usa solo los encabezados freestanding de gcc e implementa `memset`, `memcpy`, la salida serie (COM1) y una fuente de texto 5×7. La receta comprueba el artefacto con `file` (ELF 32-bit, Intel 80386) y `grub-file --is-x86-multiboot2`.

Metadatos del framebuffer. `kmain` recorre las etiquetas Multiboot2 (alineadas a 8 bytes) hasta la etiqueta de fin y obtiene el nombre del cargador, el módulo y el framebuffer. **Antes de escribir un solo píxel**, valida: tipo RGB directo, 24 o 32 bpp, dirección no nula y bajo 4 GiB, dimensiones en rango, `pitch ≥ ancho × bytes por píxel`, que la región completa quepa en 32 bits y máscaras de color coherentes. En QEMU, GRUB entregó `0xFD000000`, 800×600, pitch 3200, 32 bpp, con R/G/B en los bits 16/8/0. Cada píxel se escribe en `dirección + y·pitch + x·(bpp/8)`, con el valor armado a partir de las máscaras informadas, sin suponer el orden de los canales. El patrón de 8 barras confirma que el orden es correcto.

Conversión de la imagen. La imagen es propia: un caballo al galope de 200×150 hecho de puntos brillantes, con la misma silueta del caballo ASCII de las fases anteriores. La genera un script en Python sin azar real, así que siempre sale igual. Un PNG no es un arreglo de píxeles: está comprimido con DEFLATE y cada fila tiene un filtro predictivo. `png_a_img.py` lo decodifica (zlib más los filtros Sub, Up, Average y Paeth) y escribe el formato IMG1: una cabecera de 16 bytes (firma, ancho, alto, 3 bytes por píxel) y 90 000 bytes RGB888, fila 0 arriba. El kernel valida firma, formato, dimensiones y tamaño exacto del módulo, y dibuja a escala ×3 (600×450 en pantalla).

Pruebas. (1) La construcción genera el ELF y la ISO, y `grub-file` los acepta. (2) GRUB muestra el menú y el kernel registra por el puerto serie el magic, el cargador `GRUB 2.06-13+deb12u2` y el módulo de 90 016 bytes. (3) El patrón de prueba y la imagen aparecen con la orientación y los colores del original. (4) Con un módulo truncado a 4096 bytes, el kernel informa `TAMANO DEL MODULO NO COINCIDE CON LA CABECERA` y se detiene sin leer fuera del módulo. (5) Con `gfxpayload=text`, GRUB entrega un framebuffer de texto EGA (tipo 2, `0xB8000`, 80×25). El kernel lo rechaza para dibujar y explica el motivo en modo texto.

5. Pruebas, problemas, limitaciones y reflexión

Problemas encontrados. (a) En GRUB, `set gfxpayload=text` puesto antes de `multiboot2` no tenía efecto, porque ese comando reescribe `gfxpayload` según la cabecera del kernel; hubo que asignarlo después. (b) `ld` advertía de una pila ejecutable porque el objeto de NASM no declaraba `.note.GNU-stack`. (c) El gcc de MSYS2 no arranca desde rutas de más de 260 caracteres y genera PE. Esto, junto con la falta de GRUB en Windows, motivó el contenedor. (d) QEMU no puede abrir el registro serie (`-serial file:`) si la carpeta de destino no existe; el script de evidencias la crea antes.

Limitaciones. Fase 1: un solo sector, sin lectura de disco. Fase 2: lectura CHS de una pista (máximo 17 sectores), sin sistema de archivos ni particiones, probada solo con disquete en QEMU. Fase 3: modo fijo pedido a GRUB, solo IMG1 RGB888 sin compresión, sin interrupciones, paginación ni entrada de usuario, arranque BIOS sin UEFI. Nada se probó en hardware físico.

Reflexión y relación con la asignatura. La virtualización permitió probar cientos de arranques en segundos y sin riesgo. Una falla deja la máquina detenida, no un equipo inutilizable. Además, la máquina se controla por software: QEMU recibe teclas y entrega su pantalla por el monitor, lo que hizo posible automatizar las pruebas igual que se automatizan en un pipeline de integración continua. Las tres fases repiten el patrón de la infraestructura de datos: un artefacto inmutable (`boot_f1.bin`, `disco_f2.img`, `micro_so_f3.iso`) construido por una receta declarativa a partir de fuentes versionados y verificado antes de usarse (firmas `55AA`, `ETP2`, `IMG1`, `grub-file`). La Fase 3 lo hace explícito: el contenedor Debian fija las versiones del compilador y de GRUB, como una imagen Docker fija el entorno de un modelo o un notebook, y la misma ISO arranca igual en cualquier anfitrión con QEMU. Las validaciones del kernel ilustran otro principio: no confiar en los metadatos recibidos, sea un framebuffer o un archivo de datos, sin comprobar su coherencia.

Referencias

- QEMU Project. *QEMU System Emulation* y *QEMU Monitor*. https://www.qemu.org/docs/master/system/
- NASM. *The Netwide Assembler documentation*. https://www.nasm.us/doc/
- GNU GRUB. *Multiboot2 Specification*. https://www.gnu.org/software/grub/manual/multiboot2/multiboot.html
- GNU GRUB. *GNU GRUB Manual* (`gfxpayload`, `multiboot2`, `module2`). https://www.gnu.org/software/grub/manual/grub/
- OSDev Wiki. *Bootloader*, *BIOS*, *Bare Bones*, *Drawing In Protected Mode*. https://wiki.osdev.org/
- Ralf Brown's Interrupt List (INT 10h, 12h, 13h, 15h, 16h, 1Ah).
- W3C. *Portable Network Graphics (PNG) Specification*. https://www.w3.org/TR/png/
- GCC. *Invoking GCC* (`-ffreestanding`, `-m32`). https://gcc.gnu.org/onlinedocs/gcc/Invoking-GCC.html